This stack implementation is actually *copyable*—the copy constructor locks the mutex in the source object and then copies the internal stack. You do the copy in the constructor body ❶ rather than the member initializer list in order to ensure that the mutex is held across the copy.

As the discussion of `top()` and `pop()` shows, problematic race conditions in interfaces essentially arise because of locking at too small a granularity; the protection doesn't cover the entirety of the desired operation. Problems with mutexes can also arise from locking at too large a granularity; the extreme situation is a single global mutex that protects all shared data. In a system where there's a significant amount of shared data, this can eliminate any performance benefits of concurrency, because the threads are forced to run one at a time, even when they're accessing different bits of data. The first versions of the Linux kernel that were designed to handle multiprocessor systems used a single global kernel lock. Although this worked, it meant that a two-processor system typically had much worse performance than two single-processor systems, and performance on a four-processor system was nowhere near that of four single-processor systems. There was too much contention for the kernel, so the threads running on the additional processors were unable to perform useful work. Later revisions of the Linux kernel have moved to a more fine-grained locking scheme, so the performance of a four-processor system is much nearer the ideal of four times that of a single-processor system, because there's far less contention.

One issue with fine-grained locking schemes is that sometimes you need more than one mutex locked in order to protect all the data in an operation. As described previously, sometimes the right thing to do is increase the granularity of the data covered by the mutexes, so that only one mutex needs to be locked. However, sometimes that's undesirable, such as when the mutexes are protecting separate instances of a class. In this case, locking at the next level up would mean either leaving the locking to the user or having a single mutex that protected all instances of that class, neither of which is particularly desirable.

If you end up having to lock two or more mutexes for a given operation, there's another potential problem lurking in the wings: *deadlock*. This is